Not Rocket Science

Physical Visual Servoing Based Missile Guidance Simulator

Project Documentation

1. Repository & Hosting

- All project material – code, documentation, media, and design files – is pushed to the official RoboticsIITJ GitHub repository, `RoboticsClubIITJ/missileGNC`, on the `main` branch.
- All iteration phase items are included here, including approaches that were later replaced.
- The repository is organized into two top-level divisions plus a shared resource-guide folder, as below.
- The state of every branch before the restructure is kept as a tag: `archive/main-2026-10-05`, `archive/backup-2026-10-10` and `archive/backup-2026-09-21`.

```
/ROOT
  README.md
  learning/
    iteration-phase/
      software/
        01-aruco-detection/
        02-plant-models/
        03-first-udp-link/
        04-gimbal-board-plane-variant/
        05-simulink-versions/
        06-matlab-plotting/
      hardware/
      testing/
      reviews/
  final-product/
    raspberry-pi/
    simulink/
      plotting/
    tools/
    known-issues.md
  resource-guide/
```

Generated MATLAB and Python files (`slprj/`, `.slxc`, autosaves, `__pycache__/`, run logs) are excluded through `.gitignore` and are not treated as source.

2. Learning / Iteration Phase

This section documents the development process of the project, including experiments, alternative approaches, intermediate implementations, testing, and approaches that were later superseded. All of it lives in `learning/iteration-phase/`.

2.1 Project Evolution

The project evolved through multiple stages, beginning with individual ArUco detection experiments and progressing toward camera calibration, gimbal control, UDP communication, and Simulink integration.

- Initial ArUco detection and pose estimation.
- Development of individual plant models.
- Camera calibration and pose estimation.
- Pan – tilt servo implementation.
- Raspberry Pi to MATLAB UDP communication.
- Gimbal and ArUco integration.
- Live plotting and target streaming.
- V2 → V12 Simulink development (V12 is the final model).

Where each stage is kept in the repository:

Stage	**Folder**
ArUco detection and calibration	`software/01-aruco-detection/`
Individual plant models (Jul – Aug 2026)	`software/02-plant-models/`
First Raspberry Pi → Simulink link	`software/03-first-udp-link/`
Alternative gimbal runtime (Sep 2026)	`software/04-gimbal-board-plane-variant/`
Guidance model V2 – V11 (Aug – Oct 2026)	`software/05-simulink-versions/`
Earlier plotting and model comparison	`software/06-matlab-plotting/`

2.2 Software Approaches

2.2.1 ArUco Detection

The initial implementation used OpenCV-based ArUco detection and pose estimation on a laptop or USB webcam. Each member's experiment is kept in its own folder under `01-aruco-detection/`. The general pipeline was:

Camera
↓
ArUco Detection
↓
Pose Estimation ([x, y, z])
↓
UDP
↓
Simulink
↓
MATLAB (Plotting Trajectory)

2.2.2 Camera Calibration

A ChArUco-based calibration procedure was later introduced to obtain the camera matrix. The final version is `final-product/raspberry-pi/calibrate_camera_picamera2.py`, which writes `camera_calibration.npz`.

- Capture calibration images (at least 15 varied views of the board).
- Detect ChArUco corners.
- Match image points with board points.
- Run camera calibration and store its parameters.

2.2.3 Gimbal Control

The gimbal subsystem was developed to provide pan and tilt motion based on the detected target position. The control pipeline, as implemented in the final `servo.py`, is:

ArUco Detection
↓
Pose Estimation
↓
Angular Error (yaw $= -\text{atan2}(x, z)$, pitch $= \text{atan2}(y, \sqrt{x^2 + z^2})$)
↓
Filtering (EMA, $\alpha = 0.35$)
↓
Proportional Control (gain 0.45; I and D gains are 0)
↓
Rate Limiting (max 4° per update)
↓
Angle Limits (pan ±60°, tilt ±45°)
↓
Servo Command (1500 μs + 8 μs per degree)

2.2.4 UDP Communication

The Raspberry Pi communicates the detected target information to Simulink through UDP. The general communication architecture is:

Raspberry Pi
↓
Pose Estimation Data
↓
UDP (52-byte packet, port 25000)
↓
Simulink UDP Receiver (`GimbalUDPReceiver`, MATLAB System block)
↓
MATLAB Plotting

The first working link (`03-first-udp-link/`) streamed the pose without driving the servos. It set the 52-byte packet size that every later version kept, but put different values in the packet. Section 3.5 gives the final format.

2.2.5 Alternative Gimbal Runtime

A second version of the gimbal runtime (`04-gimbal-board-plane-variant/`, last updated 21 Sep 2026) projected the target onto the plane of the origin marker, smoothed it with an exponential moving average, sent the marker's own orientation, and rejected pose jumps above 0.15 m. The final runtime did not keep these changes. The reason it was dropped is still to be recorded in that folder's README.

2.3 Hardware Approaches

The physical system consists of the following major components:

- Raspberry Pi
- Camera (Pi Camera through Picamera2, 640 × 480)
- Pan servo (BCM GPIO 18)
- Tilt servo (BCM GPIO 19)
- Gimbal mechanism
- External servo power supply, with ground shared with the Pi
- Printed ArUco markers (`DICT_6X6_250`): ID 0 is the origin, any other ID is the target (73.8 mm edge)
- Host computer running MATLAB / Simulink

Wiring, power and a safe first power-up order are in `learning/iteration-phase/hardware/README.md`.

2.4 Testing

Testing was performed progressively as individual subsystems were completed.

1. Camera-only ArUco detection.
2. Camera calibration.
3. Marker pose estimation.
4. Servo movement.
5. Gimbal tracking.
6. UDP transmission.
7. Simulink integration.
8. End-to-end testing.

The plant and autopilot were checked against seven physics cases, kept in `testing/plant-accuracy-tests.md`: free fall, projectile motion, hover, vertical thrust, fixed deflection, sign symmetry, and an energy balance. Two tools in `final-product/tools/` test each half of the system without the other: `send_circle.py` sends a fake target moving in a circle, and `udp_test_listener.py` prints decoded packets without MATLAB.

3. Final Product

This section contains the final implementation of the project after the major experimental iterations were completed. All of it lives in `final-product/`.

3.1 Final Codebase

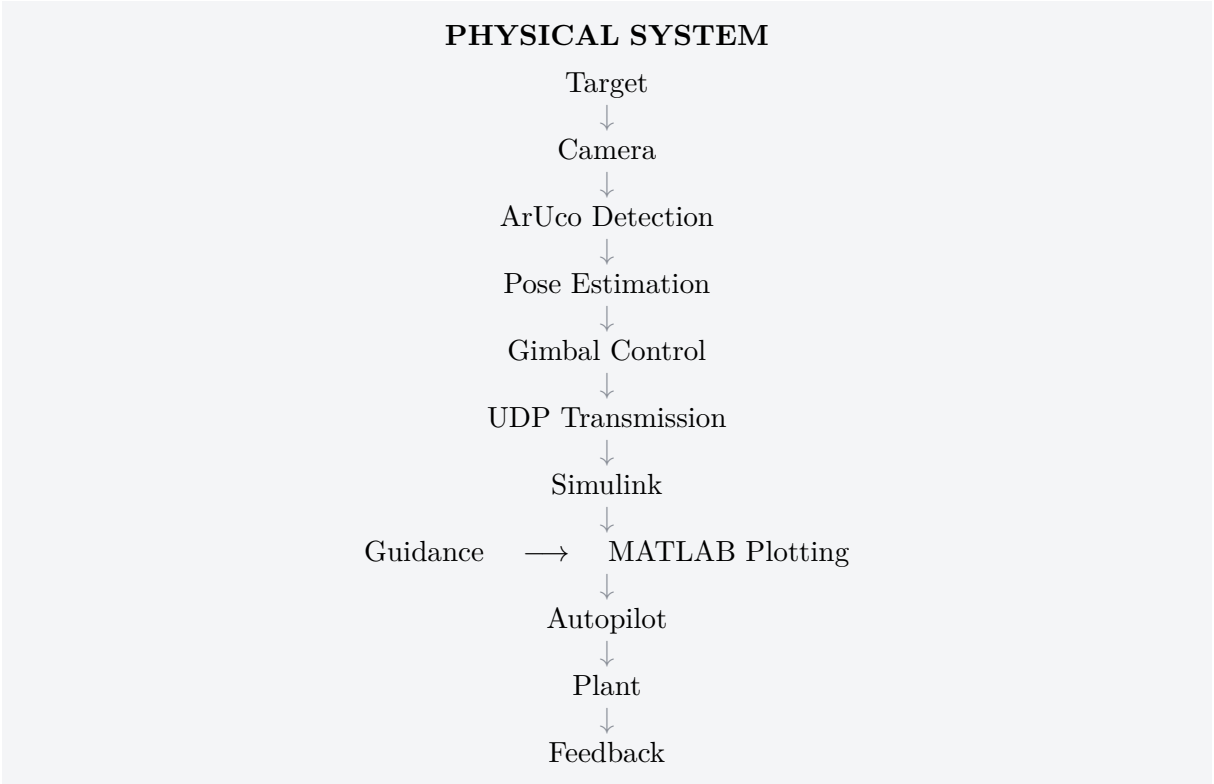
The final product contains the runtime code required for:

- Camera acquisition
- ArUco detection
- Pose estimation
- Gimbal control
- UDP transmission
- Simulink guidance
- MATLAB Plotting

File	Role
`raspberry-pi/servo.py`	Entry point on the Pi: origin search, tracking, gimbal control, UDP sender, CSV log
`raspberry-pi/aruco_gimbal_picamera2.py`	Camera and ArUco detection functions
`raspberry-pi/gimbal_config.py`	All settings; loads the calibration and `.env`
`raspberry-pi/calibrate_camera_picamera2.py`	ChArUco camera calibration
`simulink/anjaneya_V12.slx`	Guidance model
`simulink/GimbalUDPReceiver.m`	UDP receiver (MATLAB System object)
`simulink/build_udp_monitor_model.m`	Builds a small model for testing the UDP link
`simulink/plotting/`	Trajectory, live and playback plots
`tools/`	Fake target sender, packet listener, marker stage page

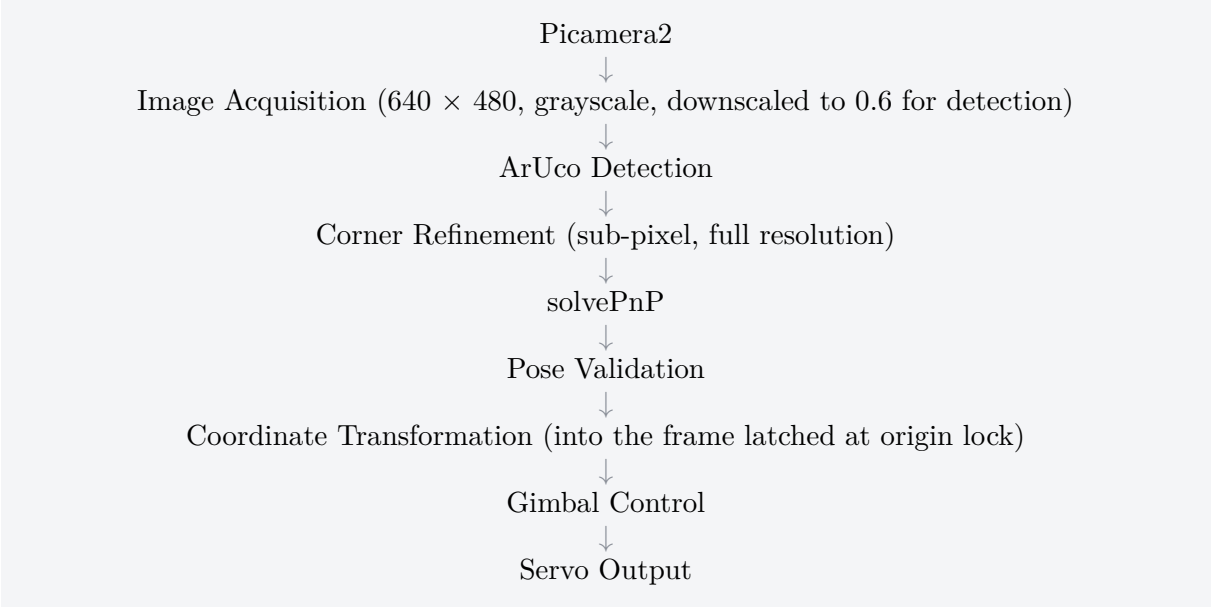
3.2 System Architecture

The overall architecture is:



3.3 Gimbal Pipeline

The gimbal subsystem follows the following pipeline:

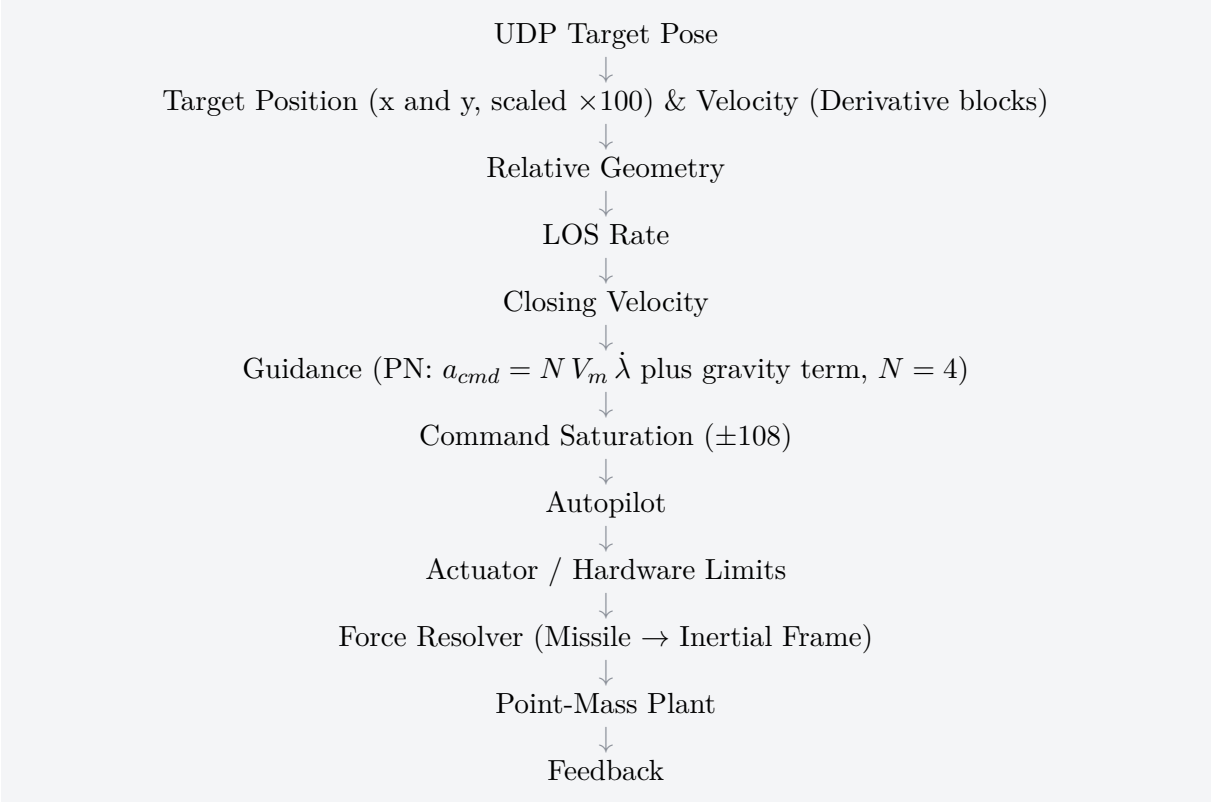


Before tracking starts, the gimbal scans in a slow raster until marker ID 0 is within 18 pixels of the image centre. Those pan and tilt angles, and the pose of ID 0, are latched as the origin. After that ID 0 is ignored, and any other marker is the target.

The pose is sent to Simulink by a separate thread at 50 Hz. It plays poses back 5 s behind real time, linearly interpolated and smoothed with a Gaussian of $\sigma = 0.3$ s.

3.4 Simulink Pipeline

The Simulink system receives target information and processes it through the guidance and plant model.



The engagement is two-dimensional, in the camera's x–y plane. The camera's z (range) is not used for guidance. A stop condition ends the run on a hit or a miss.

3.5 UDP Packet Format

Each packet is 52 bytes: six little-endian doubles followed by a `uint32` sequence number (`struct.pack("<6dI",...)`), sent to port 25000.

Slot	Sent by `servo.py`	Used in V12
1	x relative to origin (m)	Yes, ×100
2	y relative to origin (m)	Yes, ×100
3	z relative to origin (m)	Scope only
4	distance camera to target (m), labelled `bearing_pitch` in the receiver	No
5	bearing yaw (deg)	No
6	distance camera to target (m)	No

Camera axes: x positive right, y positive down, z forward.

3.6 Running the System

1. Connect the Raspberry Pi and the host computer to the same network.
2. On the host, add `final-product/simulink` to the MATLAB path and allow UDP port 25000 through the firewall.
3. The first time, test the link with `build_udp_monitor_model` and `tools/send_circle.py`; no hardware is needed.
4. On the Pi, copy `.env.example` to `.env`, set the host IP, and start `pigpiod`.
5. Start `anjaneya_V12.slx`.
6. Run `python3 servo.py`. Show marker ID 0 to lock the origin, then the target.
7. Data reaches Simulink about 5 s after the first target detection.
8. After the run, plot with `live_plot_2` or `trajectory`.

4. Known Issues and Risks

Each item below was checked against the code or the V12 model file in Oct 2026, not tested on the rig. The full list, with fixes, is in `final-product/known-issues.md`.

- **UDP packet formats.** Three meanings of the same 52-byte packet have existed: the first link (marker orientation in slots 4 and 5), the alternative gimbal runtime (board-plane position), and the final runtime. In the final one, slot 4 is labelled `bearing_pitch` but carries the distance, and bearing pitch is never computed. This has no effect on V12 guidance, which reads only slots 1 and 2.
- **Sender delay.** The final sender is 5 s behind real time. Simulink guides against where the target was 5 s earlier.
- **Multiple-target selection.** The first non-origin marker in OpenCV's detection list is tracked. That order is not guaranteed, so with two targets in view the tracked one can switch between frames. It should be a fixed ID or the nearest marker.
- **Pose validation.** The jump filter limit is 1,000,000 m, so no jump is rejected, and the error deadband is 0. These may be debugging values.
- **Older unused functions.** `pose_relative_to_marker()` and `pose_has_valid_depth()` are never called, the old 20 Hz sender is still present but commented out, and the detector's demo `main()` is not used.
- **Generated Simulink files.** These were committed before. They are now removed and listed in `.gitignore`.
- **Duplicate visualization scripts.** Four plotting scripts overlap; `live_plot_2.m` covers most of what the others do. Two different classes are both called `GimbalUDPReceiver`, so only one may be on the MATLAB path.
- **Coordinate frames.** Camera y is positive down. Whether the model treats y as up has not been checked, and the physical meaning of the $\times 100$ scale is not written down.
- **Target velocity estimation.** Velocity comes from unfiltered Derivative blocks on the scaled position, which amplifies camera noise. It should be validated against a target with known motion.

- **Model scope.** V12 is a 2-D point-mass model with gravity, thrust vectoring, actuator limits and proportional navigation. It is not an aerodynamic or 6-DOF model, and its results show the guidance loop working, not missile performance.
- **Lost target.** The receiver's `isNew` output is not used, so when the target is lost the missile keeps chasing its last known position.

5. Development Philosophy

The purpose of the documentation is not only to describe the final implementation.

It should also preserve the development journey:

What was tried
↓
Why it was tried
↓
What happened
↓
What was retained
↓
What was discarded
↓
What remains unresolved
↓
What the next team should study

Future members should therefore avoid deleting failed or superseded approaches without documentation. Such material should instead be moved into the learning / iteration archive with sufficient context to explain its purpose and outcome. Each folder there has a README answering the questions above, and `learning/iteration-phase/README.md` lists what is still to be filled in.